Git Guild 架构候选方案对比表

1. 项目背景与约束

结合 P1 阶段形成的需求规格说明书、用户故事和当前仓库现状，Git Guild 在 P2 阶段面临以下关键约束：

- 项目目标是构建一个融合代码托管、任务悬赏、成长激励与推荐能力的 Web 平台。
- 项目需要依赖现成的外部代码托管底座，不自行重做 Git 协议与底层仓库管理；本阶段技术选型优先采用 Gitea。
- 课程项目周期有限，P2 更强调架构设计的合理性、可落地性和后续实现可行性。

基于以上背景，团队选择以下两种候选架构进行对比。

2. 候选方案定义

方案 A：分层单体架构 + 外部代码托管底座集成

系统采用前后端分离方式部署：

- 前端：Vue 3 单页应用
- 后端：单个 Spring Boot 应用
- 数据库：MySQL
- 缓存：Redis（按需引入）
- 外部系统：代码托管底座（推荐 Gitea）

后端内部按照领域模块分层组织，例如：

- 用户与认证模块
- 代码托管底座适配模块
- 任务与悬赏模块
- 推荐匹配模块
- 提交与审核反馈模块
- 新手引导与项目理解模块
- 通知模块
- 成长激励模块

方案 B：微服务架构 + API 网关

系统拆分为多个独立服务，通过网关统一对外提供接口，例如：

- 网关服务
- 用户服务
- Quest 任务服务
- 成长激励服务
- 推荐服务
- 通知服务
- 仓库集成服务
- 审核反馈服务

服务间通过 HTTP 或消息机制协同，分别管理各自的数据与部署。

3. 架构候选方案对比

对比维度	方案 A：分层单体架构 + 外部代码托管底座集成	方案 B：微服务架构 + API 网关
开发复杂度	低到中。仅需维护一个后端主应用，调试、联调、部署路径清晰。	高。需要额外处理服务拆分、网关、服务通信、配置管理和部署编排。
可维护性	中到高。模块边界可在单体内部清晰划分，适合当前团队规模。	理论上较高，但前提是服务边界设计成熟；对课程团队而言运维和协作负担偏大。
可扩展性	中。后续可按模块逐步拆分现有核心模块，例如推荐、通知、统计。	高。适合大规模团队并行开发和高独立扩展需求。
性能	足以满足课程项目与中小规模演示场景；调用链更短。	在高并发和独立扩容场景下更有优势，但当前阶段收益不明显。
团队学习成本	低。与现有 `Vue 3 + Spring Boot` 技术基础一致。	高。需要掌握分布式治理、接口编排、故障处理和更复杂的 DevOps。
10 周内可行性	高。实现路径清晰，风险可控，更容易产出可演示系统。	低到中。容易把时间消耗在基础设施和联调问题上。

4. 最终建议

推荐采用：方案 A

推荐 Git Guild 在 P2 阶段采用 **分层单体架构 + 外部代码托管底座集成**。

选择理由如下：

- 当前核心挑战在于业务规则建模，而不是超大规模流量治理。
- 任务状态机、任务分类筛选、推荐匹配、管理员审核、PR 审核流转和 XP 结算等逻辑耦合较强，放在单体内更容易实现一致性。

- 现成代码托管底座可承担仓库、Issue、PR 和 Webhook 等基础能力，Git Guild 后端更适合作为业务编排与平台能力扩展层。
- 当前仓库实现仍处于早期阶段，先采用单体更符合课程项目的时间、人力和落地要求。
- 该方案保留后续演进空间，可在后续迭代中将推荐、通知、统计等现有模块逐步拆分为独立服务。

为什么不选方案 B

不选择微服务架构的主要原因如下：

- 现阶段团队规模与项目体量不足以支撑微服务带来的额外复杂度。
- 当前代码尚未形成稳定的领域边界，过早拆分服务容易造成接口反复调整。
- 任务流转、审核、奖励结算、通知等流程跨模块关联紧密，微服务会显著增加一致性处理成本。
- 课程展示更重视可实现性与完整业务闭环，而非分布式架构本身的“先进性”。

5. 初步结论

P2 阶段建议将 Git Guild 的总体结构定义为：

- 架构风格：前后端分离的分层单体架构
- 前端：Vue 3
- 后端：Spring Boot 3
- 数据库：MySQL
- 缓存：Redis（用于排行榜、热点任务等场景）
- 外部集成：Gitea API、Gitea Webhook
- 部署：Docker Compose

该结论可作为后续模块划分、接口设计、技术选型说明与 ADR 编写的基础。